

Sri Lanka Institute of Information Technology



IT3021 – Data Warehousing and Business Intelligence

Year 3

Assignment 1

Semester 1, 2026

IT Number – IT23638372

Submitted by –Dasanayaka D.M.N.N

Batch – Y3.S1.WE.DS.01.01

Contents

IT3021 – Data Warehousing and Business Intelligence	1
1.Dataset Selection	3
1.1. Description.....	3
1.2 Entity Relationship Diagram of the Instacart Grocery Basket Dataset	5
2. Preparation of the Data Sources.....	6
3. Solution Architecture	10
3.1 Data Sources	10
3.2 Data Staging	11
3.3 Data Storage (Data Warehouse)	11
3.4 ETL Process.....	11
3.5 Data Presentation (BI Layer).....	11
4.Data warehouse design & development	12
4.1. Dimensional Schema.....	12
4.2. Slowly Changing Dimensions (SCD).....	14
4.3. Assumptions.....	15
5. ETL Development.....	16
5.1. Extract Data from Source to Staging.....	16
5.2. Transform and Load Data to DW	25
6. ETL development-Accumulating Fact Tables	37

1.Dataset Selection

1.1. Description

The Instacart Online Grocery Basket Dataset contains transactional data collected through the Instacart grocery ordering and delivery app. This dataset captures customer purchasing patterns and shopping behavior, offering insights into which products are reordered, newly tried, or likely to be added to a customer's cart.

The dataset spans millions of grocery orders made by Instacart users, enabling the development of machine learning models to predict future purchases based on past behavior. This anonymized dataset was originally made public by Instacart to support the data science community and encourage innovative recommendation systems.

To align with the objectives of this project, the dataset has been reviewed and augmented with a custom-generated Customer dataset created using [Mockaroo](#) and additionally Ratings dataset done in Excel.

Six data files have been identified for this project:

1. **Orders** – Contains details about individual grocery orders placed by users.
2. **Customer** – Contains user information relevant to identifying and analyzing individual shopping behaviors.
3. **Products** – Includes information about the grocery items available in the app.
4. **Aisles** – Provides the categorization of products into aisles within the store.
5. **Departments** – Details the higher-level grouping of aisles under broader departments.
6. **Ratings** – A custom dataset including user-generated ratings for products, created using Excel and Mockaroo.

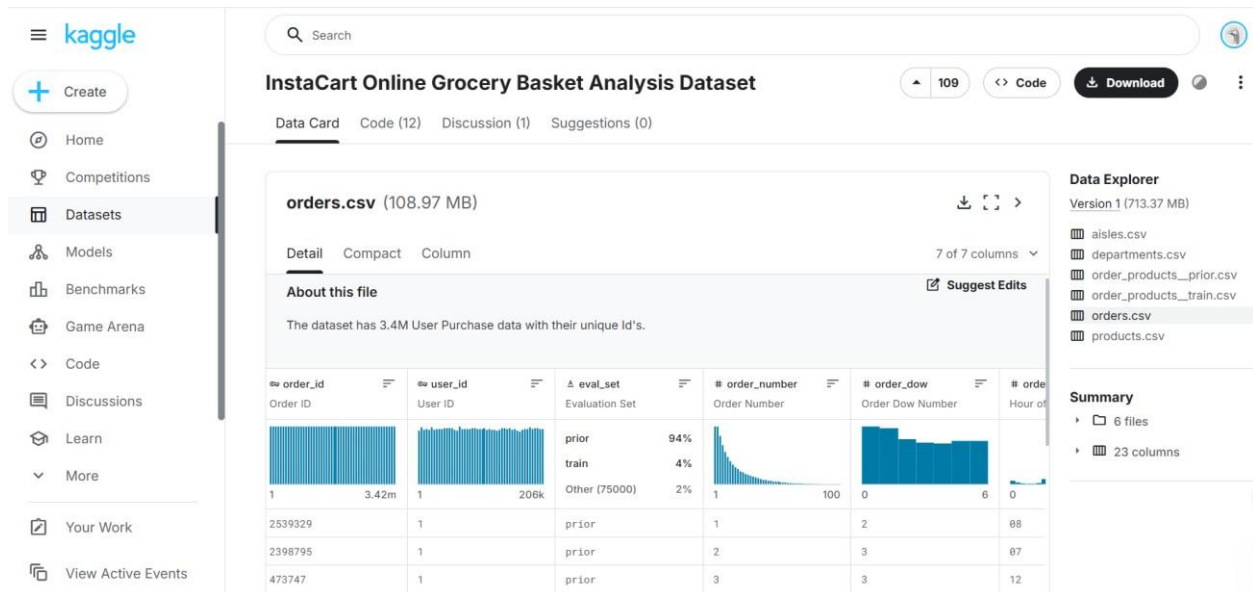
This dataset was selected because it supports:

- multiple data sources,
- a clear dimensional model,
- several ETL transformations,
- at least one slowly changing dimension,
- a date dimension, • a fact table,

sufficient records for warehouse loading and analysis.

Dataset Selected:

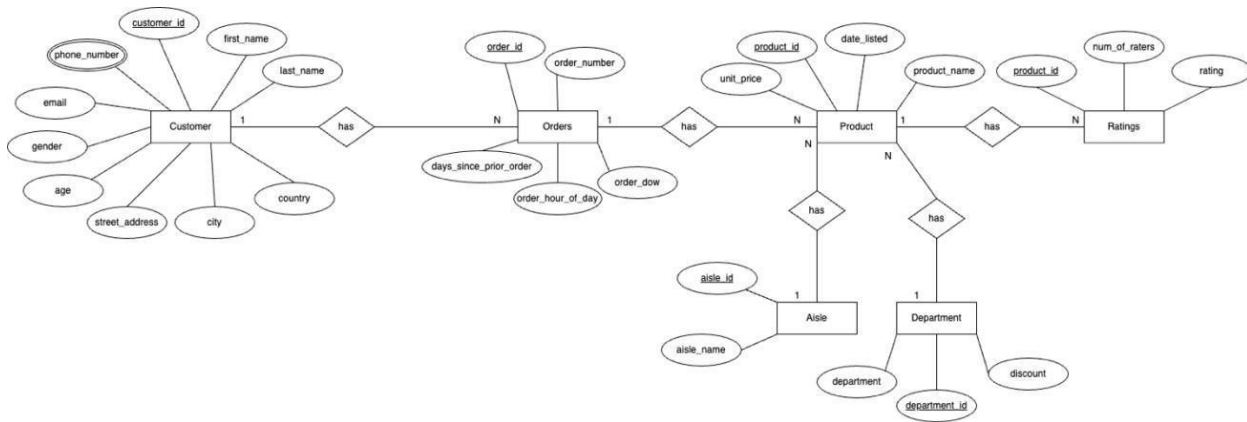
Main Dataset: <https://www.kaggle.com/datasets/yasserh/instacart-online-grocery-basketanalysis-dataset?select=orders.csv>



Custom Generated: <https://www.mockaroo.com/>

Ratings dataset was developed using Excel.

1.2 Entity Relationship Diagram of the Instacart Grocery Basket Dataset



2. Preparation of the Data Sources

initially, the original dataset files were available in CSV format. After downloading, the data was organized and structured into six separate tables. These tables were saved across different file formats to simulate diverse data sources for processing.

Three types of data sources were utilized: **.csv**, **.txt**, **.xls**, and **database .csv**

-

The **Orders**, **Order_details**, **Product** and **Aisles** data were stored in CSV format:

Orders.csv – Contains details of individual grocery orders placed by users.

Order_details.csv – Details of the orders.

Aisles.csv – Lists the categorization of products into various aisles within the store.

Products.csv– Contains information about grocery items available on the app.

.txt –

The **Ratings** data was saved as a TXT file:

Ratings.txt – A custom-generated file containing product ratings provided by users, created using Excel and Mockaroo.

Department.txt – File contains details of departments

Database –

The **Customer** data was stored in a SQL file for integration with a database system: **Customer.sql**

– Contains details of users, relevant for identifying and analyzing shopping behavior.

	A	B	C	D	E	F	G	H
1	order_id	user_id	eval_set	order_number	order_dow	order_hour_of_day	days_since_prior_order	order_date
2	2539329	1	prior	1	2	8		
3	2398795	1	prior	2	3	7	15	
4	473747	1	prior	3	3	12	21	
5	2254736	1	prior	4	4	7	29	
6	431534	1	prior	5	4	15	28	
7	3367565	1	prior	6	2	7	19	
8	550135	1	prior	7	1	9	20	
9	3108588	1	prior	8	1	14	14	
10	2295261	1	prior	9	1	16	0	
11	2550362	1	prior	10	4	8	30	
12	1187899	1	train	11	4	8	14	
13	2168274	2	prior	1	2	11		
14	1501582	2	prior	2	5	10	10	
15	1901567	2	prior	3	1	10	3	
16	738281	2	prior	4	2	10	8	
17	1673511	2	prior	5	3	11	8	
18	1199898	2	prior	6	2	9	13	
19	3194192	2	prior	7	2	12	14	
20	788338	2	prior	8	1	15	27	
21	1718559	2	prior	9	2	9	8	
22	1447487	2	prior	10	1	11	6	
23	1402090	2	prior	11	1	10	30	
24	3186735	2	prior	12	1	9	28	
25	3268552	2	prior	13	4	11	30	
26	839880	2	prior	14	3	10	13	
27	1492625	2	train	15	1	11	30	

Orders.csv

	A	B	C	D
1	order_id	product_id	add_to_cart_order	reordered
2	2	33120	1	1
3	2	28985	2	1
4	2	9327	3	0
5	2	45918	4	1
6	2	30035	5	0
7	2	17794	6	1
8	2	40141	7	1
9	2	1819	8	1
10	2	43668	9	0
11	3	33754	1	1
12	3	24838	2	1
13	3	17704	3	1
14	3	21903	4	1
15	3	17668	5	1
16	3	46667	6	1
17	3	17461	7	1
18	3	32665	8	1
19	4	46842	1	0
20	4	26434	2	1
21	4	39758	3	1
22	4	27761	4	1
23	4	10054	5	1
24	4	21351	6	1
25	4	22598	7	1
26	4	34862	8	1
27	4	40285	9	1

order_details (+)

Order_details.csv

	A	B	C	D
	product_id	product_name	aisle_id	department_id
1	1	Chocolate Sandwich Cookies	61	19
2	2	All-Seasons Salt	104	13
3	3	Robust Golden Unsweetened Oolong Tea	94	7
4	4	Smart Ones Classic Favorites Mini Rigatoni With Vodka Cream Sauce	38	1
5	5	Green Chile Anytime Sauce	5	13
6	6	Dry Nose Oil	11	11
7	7	Pure Coconut Water With Orange	98	7
8	8	Cut Russet Potatoes Steam N' Mash	116	1
9	9	Light Strawberry Blueberry Yogurt	120	16
10	10	Sparkling Orange Juice & Prickly Pear Beverage	115	7
11	11	Peach Mango Juice	31	7
12	12	Chocolate Fudge Layer Cake	119	1
13	13	Saline Nasal Mist	11	11
14	14	Fresh Scent Dishwasher Cleaner	74	17
15	15	Overnight Diapers Size 6	56	18
16	16	Mint Chocolate Flavored Syrup	103	19
17	17	Rendered Duck Fat	35	12
18	18	Pizza for One Suprema Frozen Pizza	79	1
19	19	Gluten Free Quinoa Three Cheese & Mushroom Blend	63	9
20	20	Pomegranate Cranberry & Aloe Vera Enrich Drink	98	7
21	21	Small & Medium Dental Dog Treats	40	8
22	22	Fresh Breath Oral Rinse Mild Mint	20	11
23	23	Organic Turkey Burgers	49	12
24	24	Tri-Vi-Sol® Vitamins A-C and D Supplement Drops for Infants	47	11
25	25	Salted Caramel Lean Protein & Fiber Bar	3	19
26	26	Fancy Feast Trout Feast Flaked Wet Cat Food	41	8

products (+)

Products.csv

	A	B
1	aisle_id	aisle
2		1 prepared soups salads
3		2 specialty cheeses
4		3 energy granola bars
5		4 instant foods
6		5 marinades meat preparation
7		6 other
8		7 packaged meat
9		8 bakery desserts
10		9 pasta sauce
11		10 kitchen supplies
12		11 cold flu allergy
13		12 fresh pasta
14		13 prepared meals
15		14 tofu meat alternatives
16		15 packaged seafood
17		16 fresh herbs
18		17 baking ingredients
19		18 bulk dried fruits vegetables
20		19 oils vinegars
21		20 oral hygiene
22		21 packaged cheese
23		22 hair care
24		23 popcorn jerky
25		24 fresh fruits
26		25 soap
27		26 coffee

Aisles.csv


```

department_id,department
1,frozen
2,other
3,bakery
4,produce
5,alcohol
6,international
7,beverages
8,pets
9,dry goods pasta
10,bulk
11,personal care
12,meat seafood
13,pantry
14,breakfast
15,canned goods
16,dairy eggs
17,household
18,babies
19,snacks
20,deli
21,missing

```

Department.txt

product_id	num_of_rates	rating
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7
8	8	8
9	9	9
10	10	10
11	11	11
12	12	12
13	13	13
14	14	14
15	15	15
16	16	16
17	17	17
18	18	18
19	19	19
20	20	20
21	21	21
22	22	22
23	23	23
24	24	24
25	25	25

Ratings.txt

Object Explorer

Connect

Databases

System Databases

Database Snapshots

Customer_Details

InstaCart_Grocery_Basket_DW

Database Diagrams

Tables

System Tables

FileTables

External Tables

Graph Tables

dbo.DimAisle

dbo.DimCustomer

dbo.DimDate

dbo.DimDepartment

dbo.DimProduct

dbo.DimRating

dbo.FactOrder

Columns

order_id (int, null)

customer_SK (int, null)

product_SK (int, null)

dateSK (int, null)

unit_price (float, null)

discount (float, null)

selling_price (float, null)

accm_bn_create_time (datetime, null)

Customer.sql

Customersq...Nethmi (571)

SQL Query1.sq...Nethmi (641*)

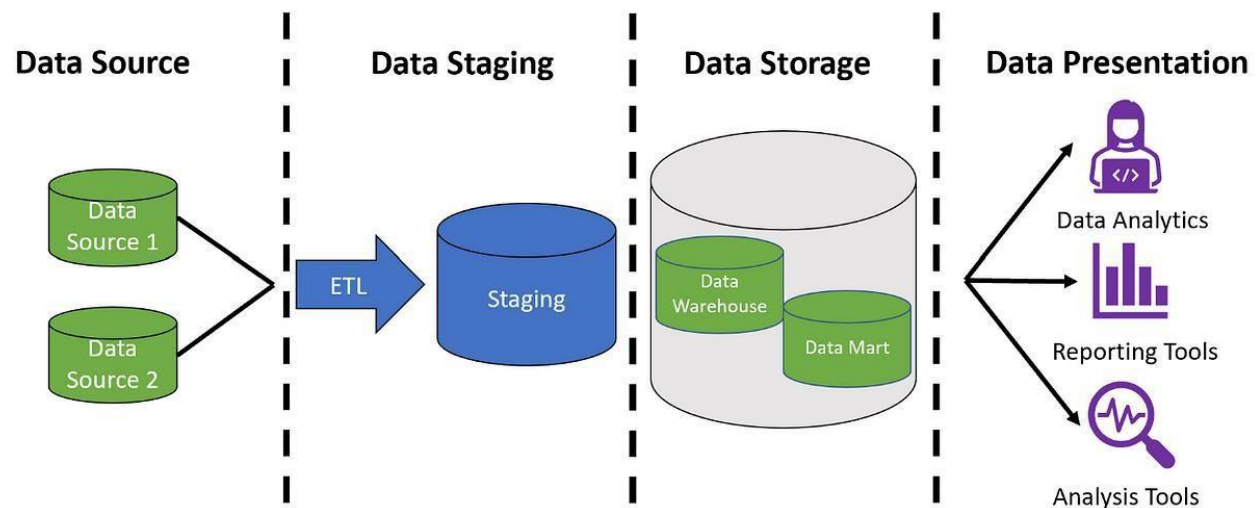
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Customer.sql

3. Solution Architecture



The diagram above illustrates a high-level Data Warehousing and Business Intelligence (BI) architecture tailored for the **Instacart Online Grocery Basket Analysis** dataset. This architecture can be broken down into four main layers:

3.1 Data Sources

The first step in any data warehousing solution is gathering data from multiple sources, which may include databases, flat files, APIs, or external systems, depending on the needs and context of the project.

In this scenario, two types of data sources were used:

- **Database Source:**

- Customer.sql

- **Flat Files (CSV and TXT):**

- Orders.csv

- Aisles.csv

- Products.csv

- Departments.txt

- Ratings.txt

3.2 Data Staging

A separate **staging database** was created to temporarily hold extracted data before transformation and loading into the warehouse. The purpose of the staging area was to:

- isolate raw data from warehouse tables,
- support data cleansing and validation,
- perform datatype correction,
- reduce direct dependency on source files,
- and simplify ETL execution.

This staging layer was especially useful because the project used multiple source formats and required several conversion and lookup operations.

3.3 Data Storage (Data Warehouse)

A **data warehouse** is a centralized repository that stores integrated data from multiple sources. It supports historical analysis and business reporting.

For the Instacart project:

- A warehouse structure was created to house all cleaned and integrated data.
- This included dimension tables such as Products, Aisles, Departments, Customers, and Orders, and a fact table capturing transactional insights and customer behavior

3.4 ETL Process

ETL stands for **Extract, Transform, and Load**, and is a crucial process in integrating data from various sources into the warehouse.

In this scenario:

- **Extract:** Data is pulled from CSV, TXT, and SQL files.
- **Transform:** Cleaning, normalization, joining, and aggregation of datasets.
- **Load:** Final structured data is stored in the warehouse for analytics and BI use

3.5 Data Presentation (BI Layer)

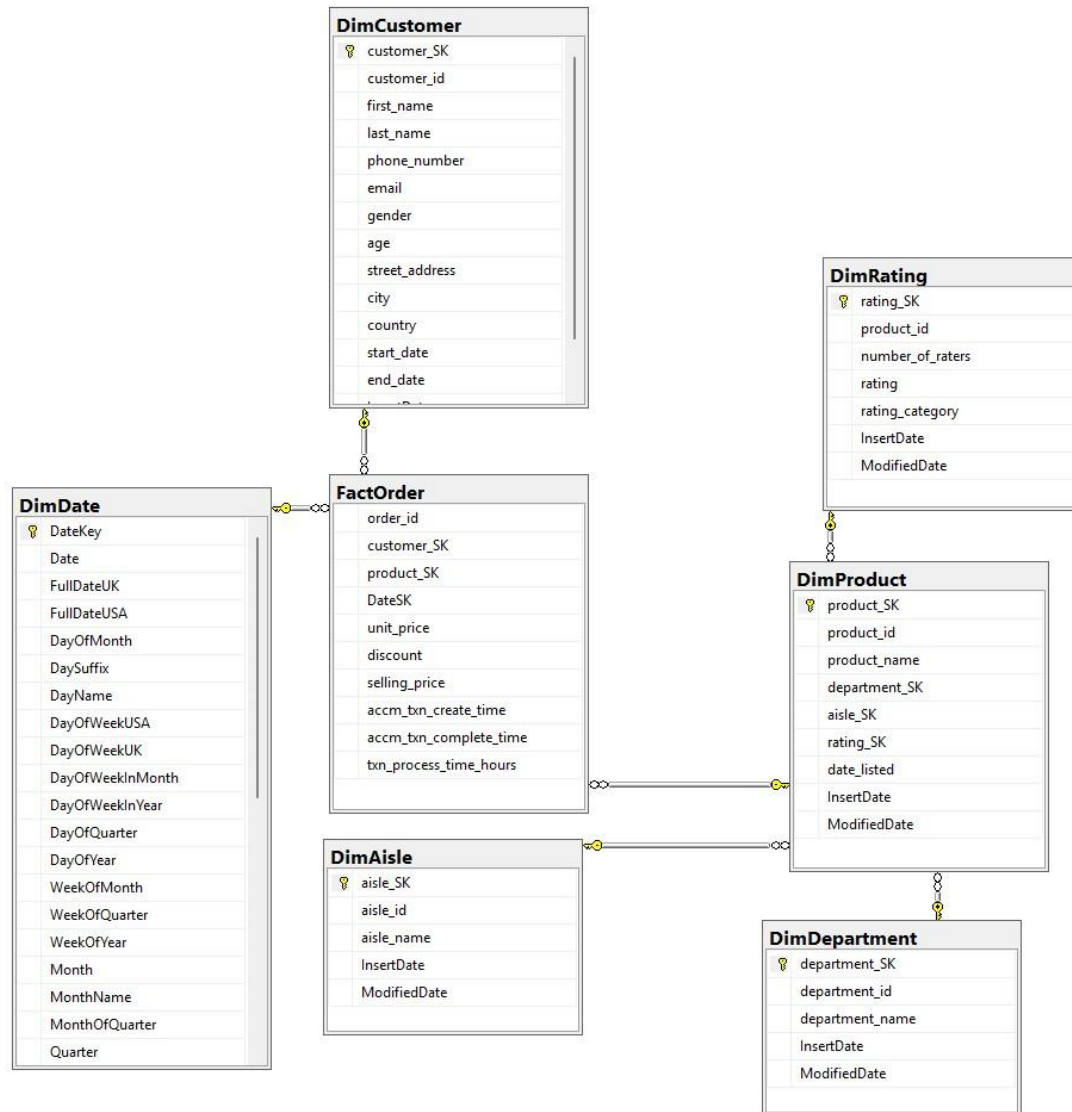
The final layer includes tools and technologies that transform raw data into actionable insights for business users, analysts, and stakeholders.

This can involve:

- **Data Analytics** – Running predictive models to identify products users are likely to reorder.
- **Reporting Tools** – Visual dashboards and summary reports on purchasing patterns.
- **Analysis Tools** – Detailed queries and explorations of customer behavior, product popularity, and department performance.

4.Data warehouse design & development

4.1. Dimensional Schema



A **snowflake schema** was used for this project. This approach was selected because the Instacart dataset contains natural hierarchical relationships between products, aisles, and departments. Instead of storing all descriptive product category information directly in one denormalized table, separate dimensions were maintained to preserve structure and improve clarity.

After identifying the facts and dimensions there are 8 tables in the data warehouse.

The warehouse consists of:

- **FactOrder** – fact table
- **DimProduct** – dimension table
- **DimDepartment** – dimension table
- **DimAisle** – dimension table
- **DimRating** – dimension table
- **DimCustomer** – dimension table
- **DimDate** – common dimension table

1. **FactOrder** – Holds all of the sales information of the transactions. It uses the dimension tables through foreign keys
2. **DimProduct** – The product dimension holds detailed product information. Product_SK is the surrogate key
3. **DimDepartment** – This dimension holds data regarding the product categorization by departments. Department_SK serves as the surrogate key.
4. **DimAisle** – The aisle dimension holds data on the product categorization by aisle. The surrogate key is Aisle_SK
5. **DimRating** -Stores product rating information and rating category.
6. **DimCustomer** – The customer dimension holds customer details. The surrogate key is Customer_SK.
7. **DimDate** – It's a typical date dimension accommodating time-based analysis. The surrogate key is DateKey. The date dimension was created using an SQL script.

4.2. Slowly Changing Dimensions (SCD)

It was assumed that certain dimensions may change over time. Therefore, some dimensions are implemented as **Slowly Changing Dimensions (Type 2)**.

DimCustomer

DimCustomer was implemented using both **Type 1** and **Type 2** SCD behavior.

- **Changing Attributes (Type 1 SCD)**

These values are updated directly when changes occur:

age
first_name
last_name
phone_number

- **Historical Attributes (Type 2 SCD) T**

These values are tracked by inserting new rows to preserve history:

city
country
email
street_address

For Type 2 handling, historical tracking columns were included:

start_date end_date
is_current

DimRating

DimRating was implemented using a **Type 1 SCD** approach, where:

rating
rating_category

are treated as changing attributes and overwritten when new values are received

4.3. Assumptions

- Customer dimension and Rating dimension are slowly changing dimensions and they keep history records.
- Product names can't be updated.
- Department names are not changing
- Address dimension contains all possible addresses.
- Extended string data types were used to suite future data

5. ETL Development

5.1. Extract Data from Source to Staging

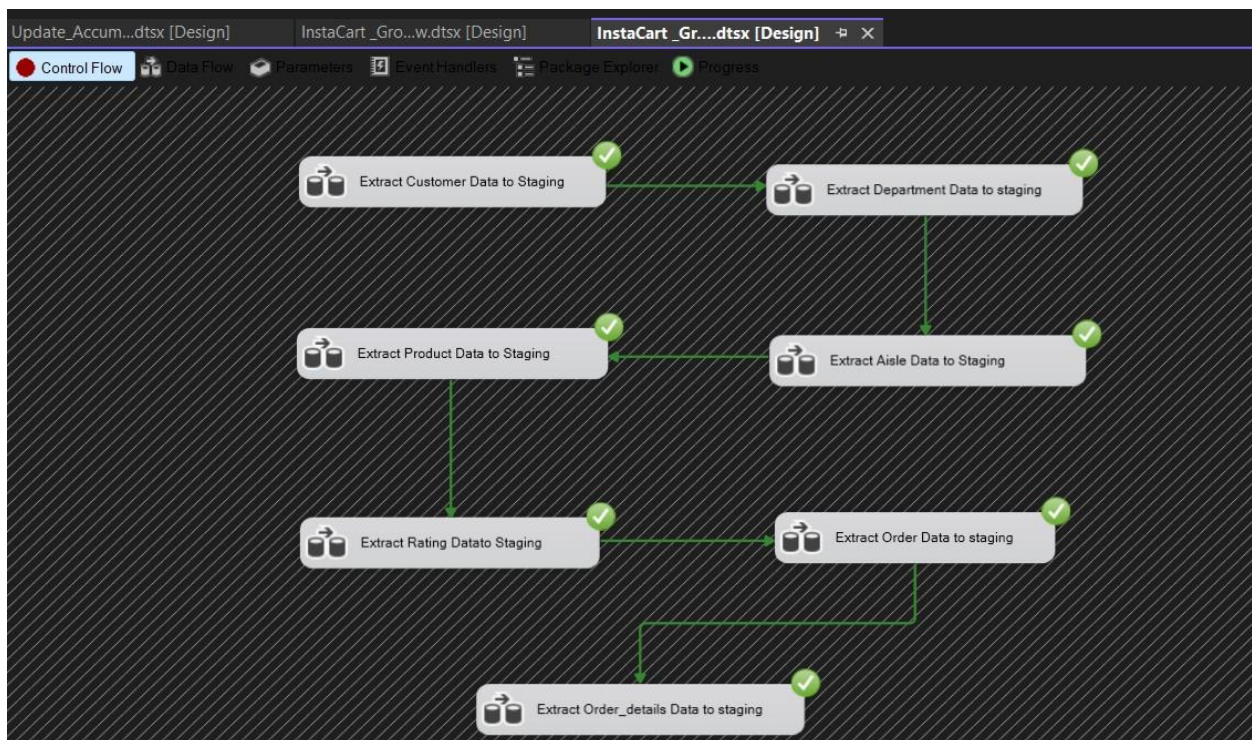
Initially, all data was extracted from the various data sources and loaded into dedicated **staging tables** to ensure a clean and structured transition into the data warehouse. This approach helps in isolating raw data from the core processing environment and provides a buffer zone for data validation, transformation, and cleansing.

For this purpose, **SQL Server Integration Services (SSIS)** was utilized as the ETL (Extract, Transform, Load) tool.

The data extraction process was implemented using **Control Flow** tasks in SSIS, allowing data to be systematically pulled from the following sources:

- **CSV files** (Orders.csv, Aisles.csv, Product.csv)
- **TXT file** (Ratings.txt, Departments.csv))
- **SQL script-based database file** (Customer.sql)

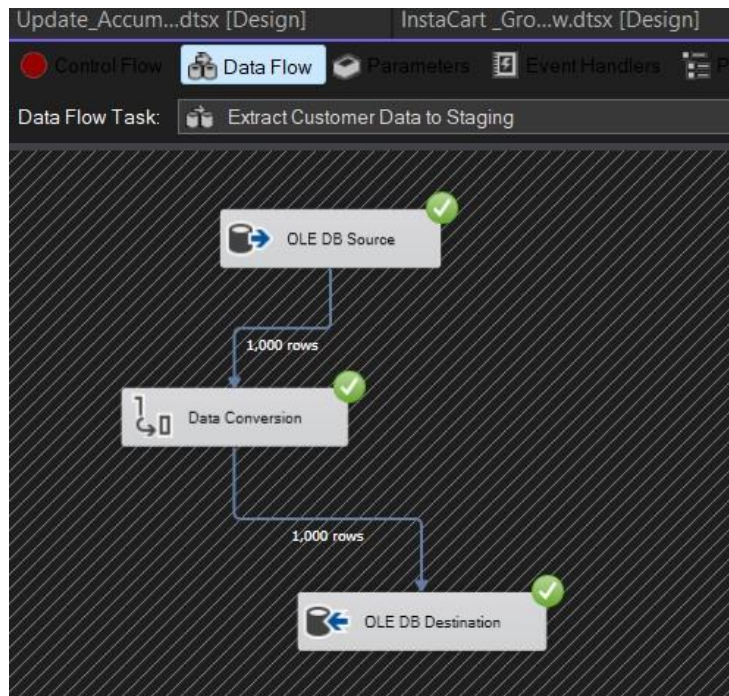
Each of these sources was connected to corresponding **staging tables** in the staging database



5.1.1. Data flow of extracting Customer Data to staging

The **Customer table** from the source database is extracted using an **OLE DB Source**. A **Data Conversion** transformation is applied to convert columns like first_name, last_name, phone_number, email, etc., into the required **Unicode string (DT_WSTR)** format to match the schema of the staging table.

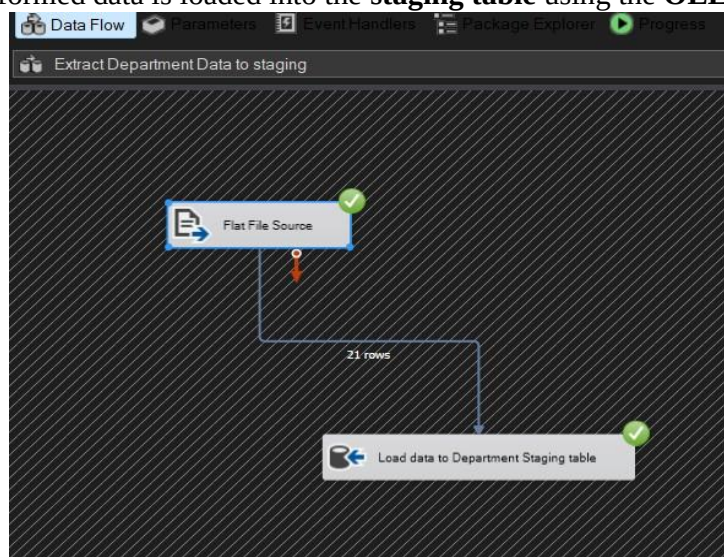
Finally, the converted data is loaded into the **staging table** using the **OLE DB Destination**.



5.1.2. Data flow of extracting Department Data to staging

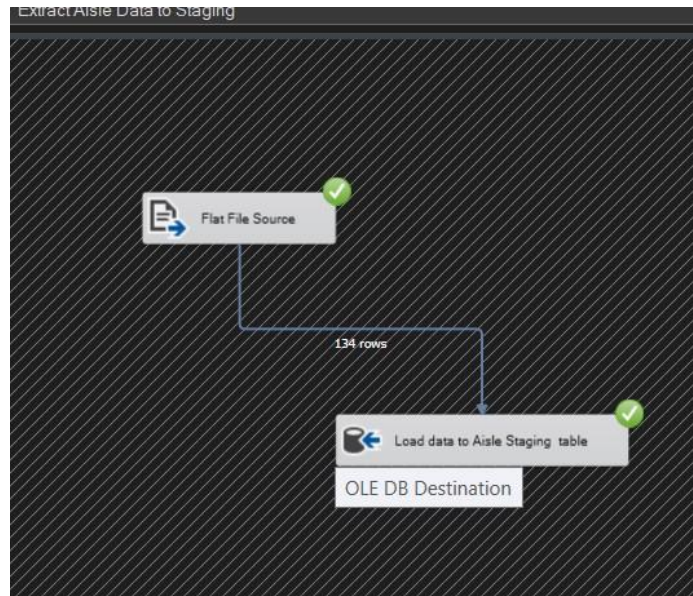
The **Departments data** is extracted from a **flat file source (CSV file)**.

Finally, the transformed data is loaded into the **staging table** using the **OLE DB Destination**



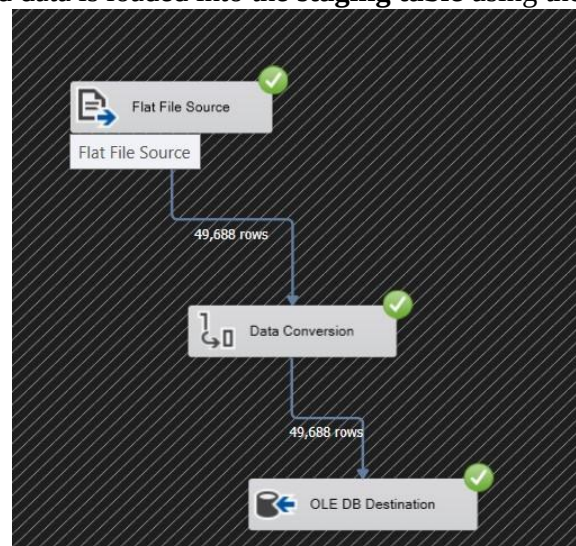
5.1.3. Data flow of extracting Aisle Data to staging

The **Aisle data** is extracted from a **flat file source (CSV file)**.
Finally, the transformed data is loaded into the **staging table** using the



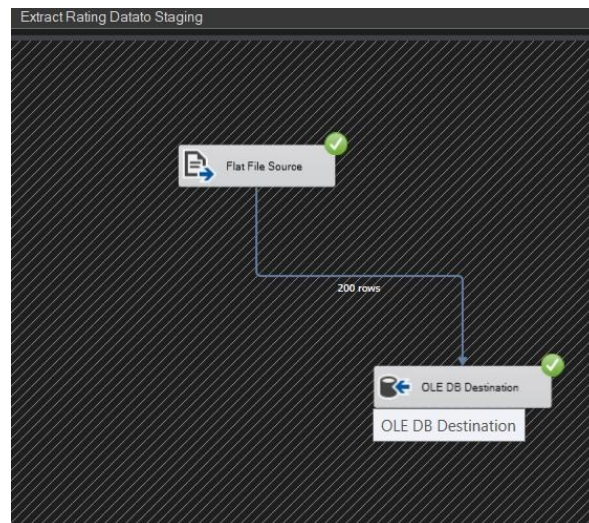
5.1.4. Data flow of extracting Product Data to staging

The **Products data** is extracted from a **flat file source (CSV file)**.
A **Data Conversion** transformation is applied to convert the product_name column into **Unicode string (DT_WSTR)** with a length of 255 to ensure compatibility with the staging table schema.
Finally, the converted data is loaded into the **staging table** using the **OLE DB Destination**.



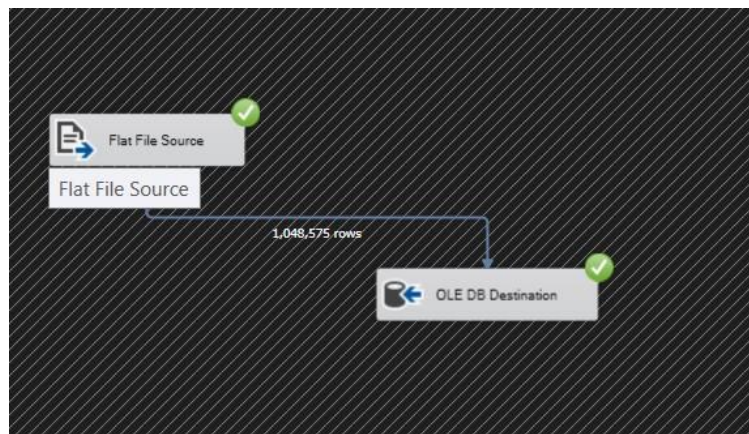
5.1.5. Data flow of extracting Rating Data to staging

The **Rating data** is extracted from a **flat file source (txt file)**.
Finally, the transformed data is loaded into the **staging table** using the



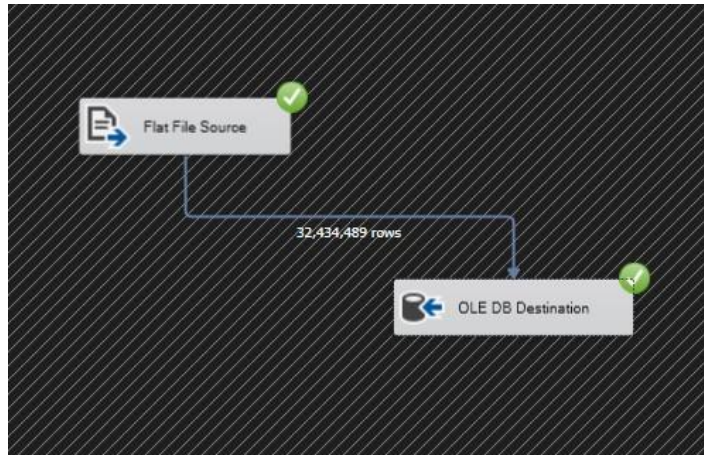
5.1.6. Data flow of extracting Order Data to staging

The **Order data** is extracted from a **flat file source (csv file)**.
Finally, the transformed data is loaded into the **staging table** using the **OLE DB Destination**



5.1.7. Data flow of extracting Order_details Data to staging

The **Order_details data** is extracted from a **flat file source (csv file)**. Finally, the transformed data is loaded into the **staging table** using the



5.1.8. Preventing Duplicate Loads in Staging Tables

When the SSIS package is executed multiple times, the **staging tables** would normally accumulate duplicate records because data is loaded from the source **without removing existing rows**. To prevent this, **OnPreExecute event handlers** were configured for each **Data Flow Task**.

Control Flow Data Flow Parameters Event Handlers Package Explorer

Executable: Extract Department to staging Event handler: OnPreExecute Delete

Truncate Department

Control Flow Data Flow Parameters Event Handlers Package Explorer

Executable: Extract Order to staging Event handler: OnPreExecute Delete

Truncate Order

Control Flow Data Flow Parameters Event Handlers Package Explorer

Executable: Extract Order_details to staging Event handler: OnPreExecute Delete

Truncate Order_details

Executable: Extract Product to staging Event handler: OnPreExecute Delete

Truncate Product

Executable: Extract Aisles to staging Event handler: OnPreExecute Delete

Truncate Aisles

Control Flow Data Flow Parameters Event Handlers Package Explorer

Executable: Extract Rating to stage Event handler: OnPreExecute Delete

Truncate Rating

Control Flow Data Flow Parameters Event Handlers Package Explorer

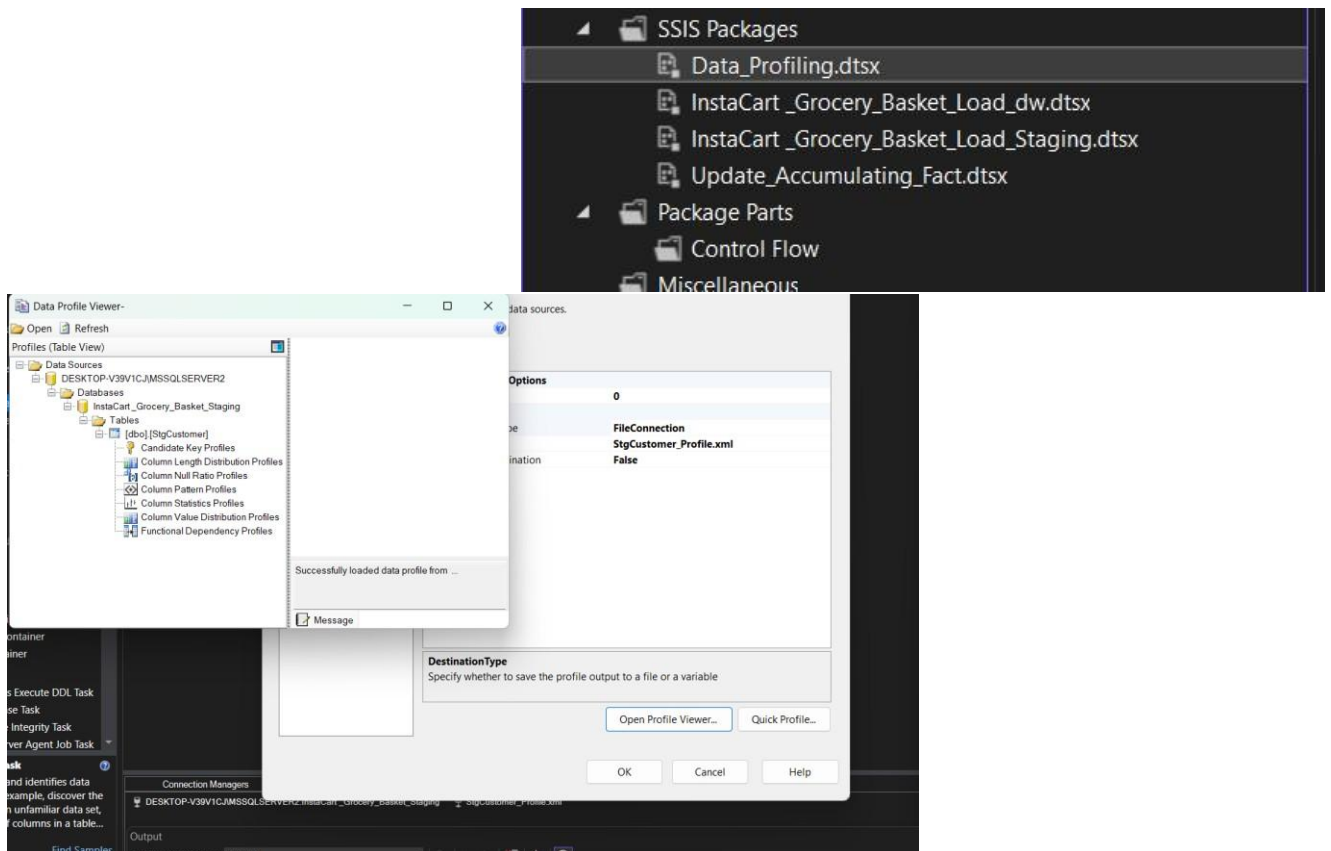
Executable: Extract Customer to staging Event handler: OnPreExecute Delete

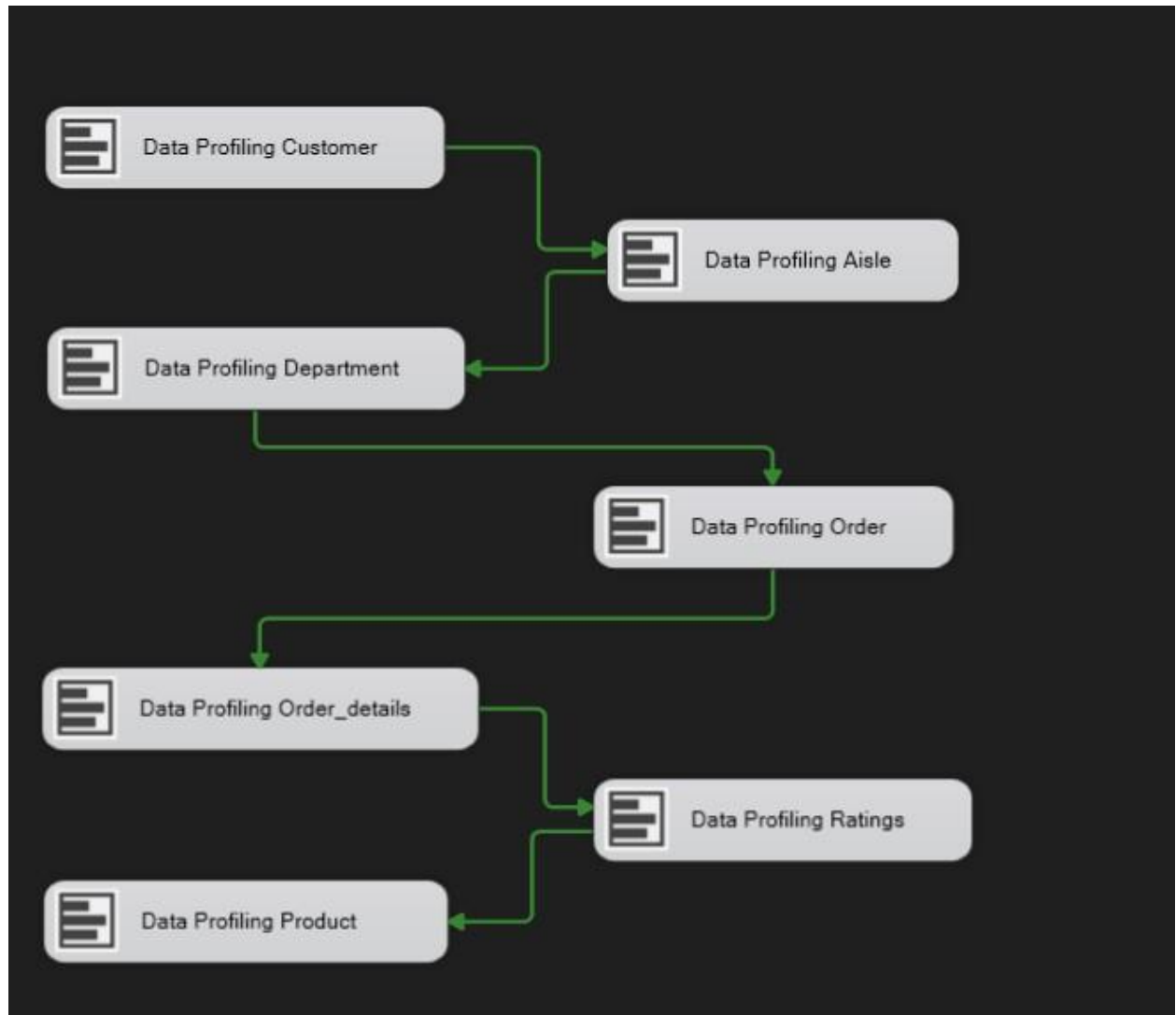
Truncate Customer Staging Table

5.1.9. Data Profiling

Data profiling was done to obtain an overview of the data. Data types, count and null values were identified from this process.

- Right click on SSIS Packages and New SSIS Packages – DataProfiling.dtsx • Drag and drop the Data Profiling Task and double click to open configuration.
- Click on Quick Profile button to open Quick Profile Form.
- Click on new button and create the connection.
- Once the green tick appeared, double click the Data Profiling Task and click on the Open profile viewer to view the analyzed data.





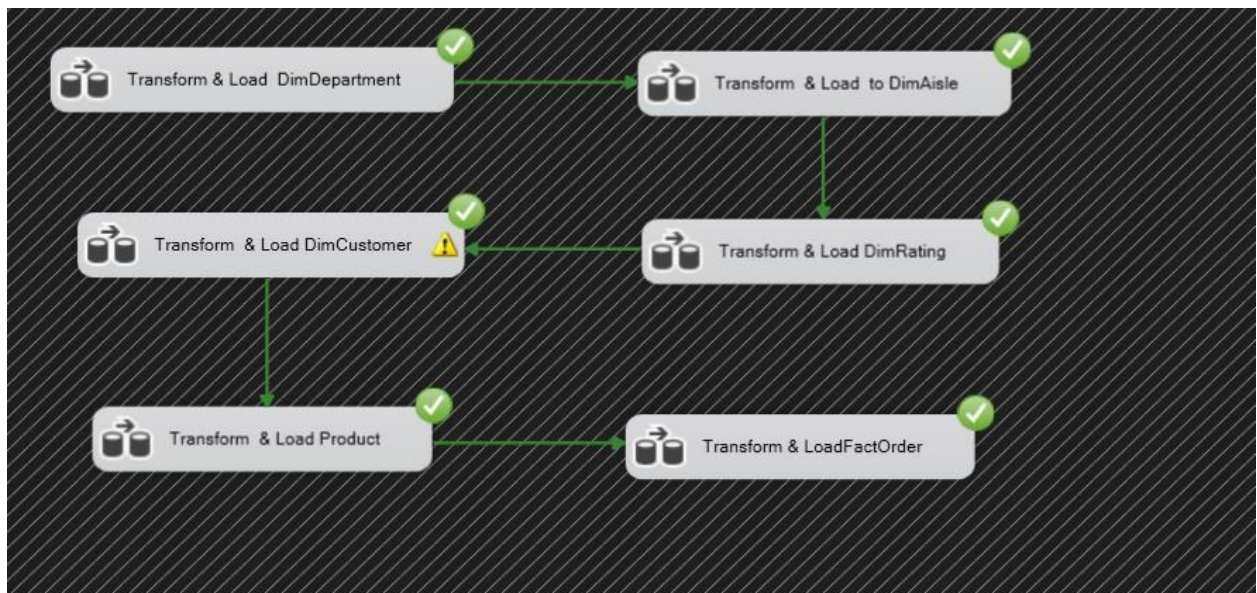
5.2. Transform and Load Data to DW

Once the data was staged, it was transformed and loaded into the **data warehouse** using a **Snowflake Schema** design. This schema consists of **five dimension tables** and **one fact table**. The structure was implemented in SQL Server, and **stored procedures** were used to ensure data integrity and prevent redundancy.

The Instacart_Grocery_Load_DW.dtsx package controls the data warehouse loading. It runs tasks in a set order to maintain data integrity.

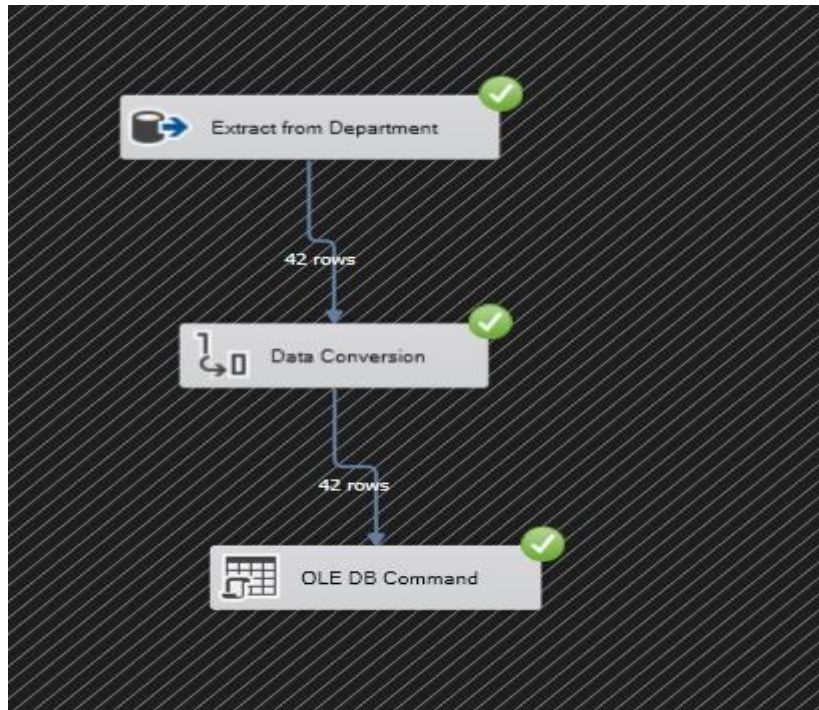
A **Date Dimension** was also created and populated using a custom **SQL procedure**.

The SSIS workflow shown above illustrates the transformation and load process from the staging tables to the warehouse.



5.2.1. Transform & Load to DimDepartment

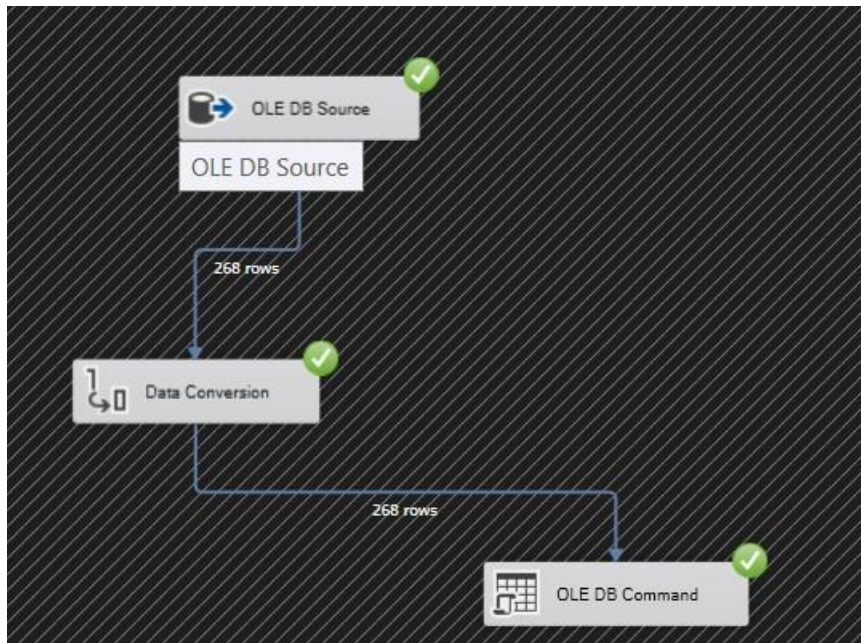
A **Data Conversion transformation** was applied to the *department* attributes to resolve data type mismatches between the staging tables and dimension tables. Specifically, these fields were converted to a compatible data type (e.g., from non-Unicode to Unicode string or vice versa) to ensure successful lookup operations.



```
CREATE OR ALTER PROCEDURE dbo.UpdateDimDepartment
    @department_id INT,
    @department_name NVARCHAR(50)
AS
BEGIN
    IF NOT EXISTS (
        SELECT department_SK
        FROM dbo.DimDepartment
        WHERE department_id = @department_id
    )
    BEGIN
        INSERT INTO dbo.DimDepartment (department_id, department_name)
        VALUES (@department_id, @department_name);
    END
    ELSE
    BEGIN
        UPDATE dbo.DimDepartment
        SET department_name = @department_name
        WHERE department_id = @department_id;
    END
END;
```

5.2.2. Transform & Load to DimAisle

A **Data Conversion transformation** was applied to the ***department*** attributes to resolve data type mismatches between the staging tables and dimension tables. Specifically, these fields were converted to a compatible data type (e.g., from non-Unicode to Unicode string or vice versa) to ensure successful lookup operations.



```

CREATE OR ALTER PROCEDURE dbo.UpdateDimAisle
    @aisle_id INT,
    @aisle_name NVARCHAR(100)
AS
BEGIN
    IF NOT EXISTS (
        SELECT aisle_SK
        FROM dbo.DimAisle
        WHERE aisle_id = @aisle_id
    )
    BEGIN
        INSERT INTO dbo.DimAisle (aisle_id, aisle_name)
        VALUES (@aisle_id, @aisle_name);
    END

    ELSE
    BEGIN
        UPDATE dbo.DimAisle
        SET aisle_name = @aisle_name
        WHERE aisle_id = @aisle_id;
    END
END;

```

5.2.3. Transform & Load to DimRating

The transformation and load process for the **DimRating** table is designed to handle **Slowly Changing Dimensions (SCD)** using SSIS.

Derived Column – rating_category

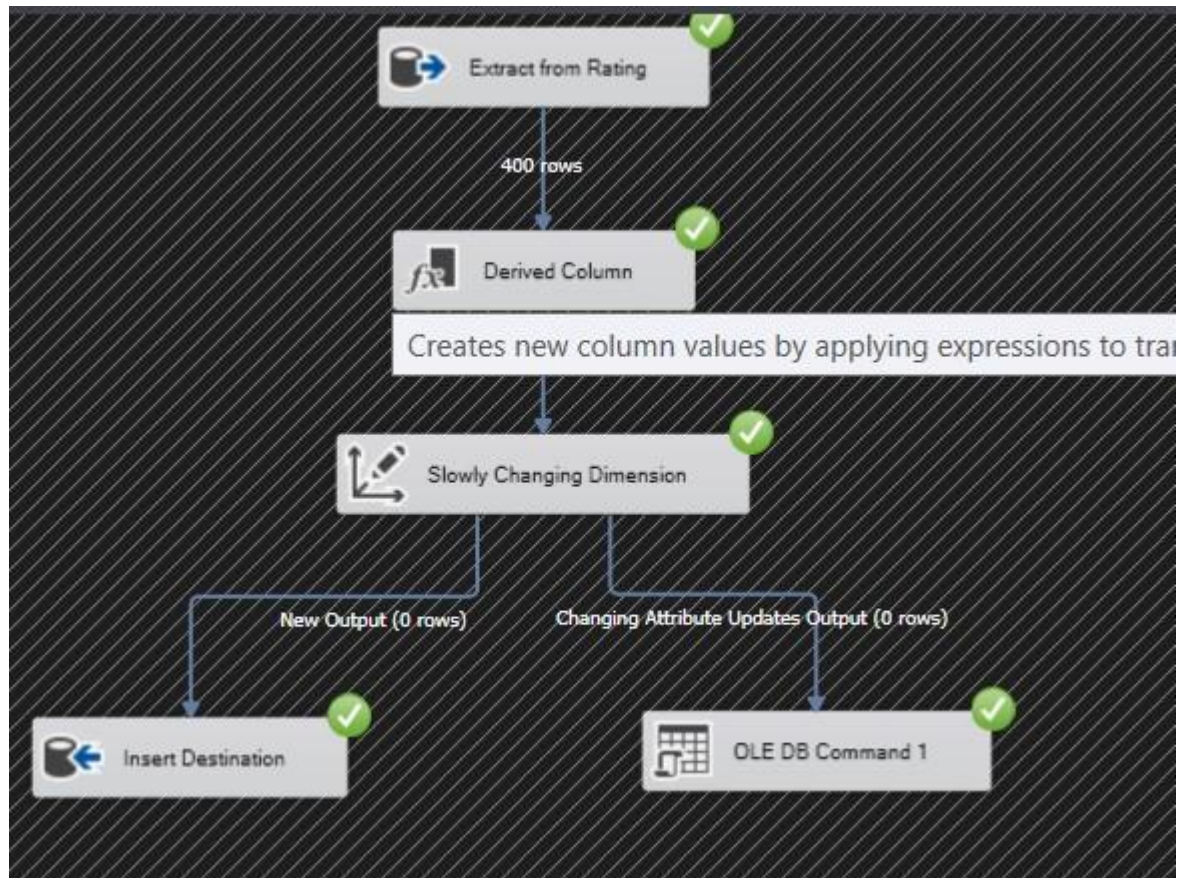
A **derived column** named **rating_category** was created using an SSIS expression to classify customer ratings into categories. The expression used is:

- If rating is **greater than or equal to 4.5**, the category is "**Excellent**".
- If rating is **greater than or equal to 3 but less than 4.5**, the category is "**Average**".
- If rating is **less than 3**, the category is "**Poor**".

Slowly Changing Dimension (SCD) Component •

Manages updates to dimension data.

- Both rating and rating_category columns are configured as **changing attributes (Type 1 SCD)**, meaning their updates overwrite existing values.



Derived Column Transformation Editor

Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.

Variables and Parameters
Columns

Mathematical Functions
String Functions
Date/Time Functions
NULL Functions
Type Casts
Operators

Description:

Derived Column Name	Derived Column	Expression	Data Type
rating_category	<add as new column>	rating >= 4.5 ? "Excellent" : rating >= 3 ? "Ave..."	Unicode string [DT_...]

Configure Error Output... OK Cancel Help

Slowly Changing Dimension Wizard

Select a Dimension Table and Keys

Select a dimension table to load and map columns in the transformation input to

Connection manager:
DESKTOP-V39V1CJ_MSSQLSERVER2.Instacart_Grocery_Basket_DW New...

Table or view:
[dbo].[DimRating]

Input Columns	Dimension Columns	Key Type
num_of_raters	number_of_raters	Not a key column
product_id	product_id	Business key
rating	rating	Not a key column
rating_categ...	rating_category	Not a key column

Help < Back Next > Finish >>| Cancel

5.2.4. Transform & Load to DimCustomer

The transformation and load process for the **DimCustomer** table handles both **changing** and **historical** attributes using the **Slowly Changing Dimension (SCD)** component in SSIS.

In the **Slowly Changing Dimension Wizard**, the following settings were applied: •

Changing Attributes (Type 1 SCD)

These values are updated directly when changes occur:

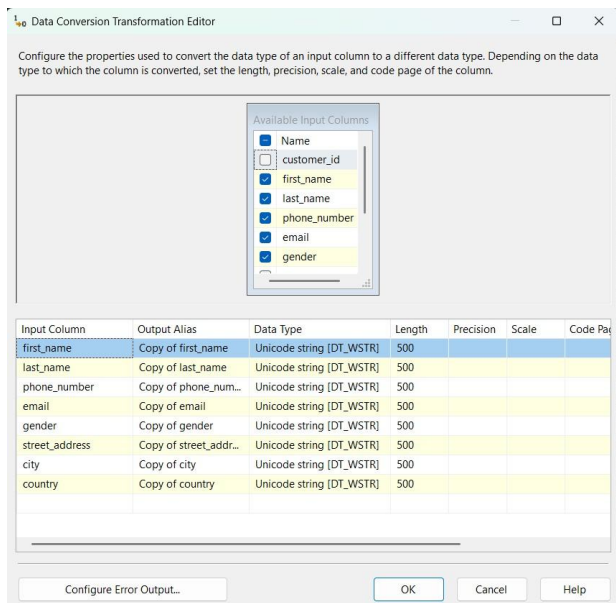
age
first_name
last_name
phone_number

- **Historical Attributes (Type 2 SCD)**

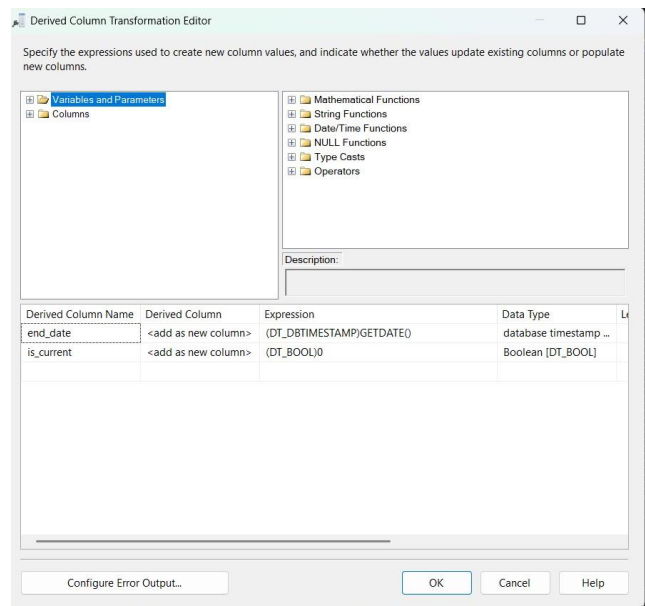
These values are tracked by inserting new rows to preserve history:

city
country
email
street_address

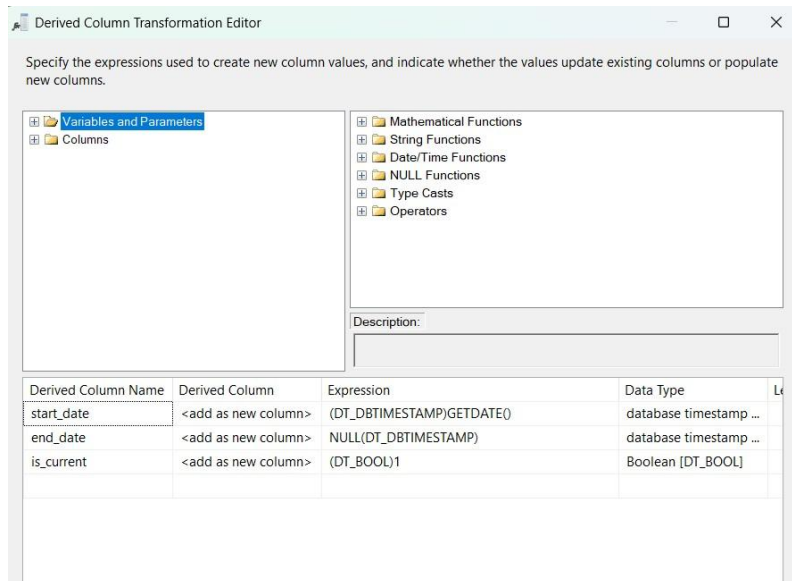




Data convention



Derived column

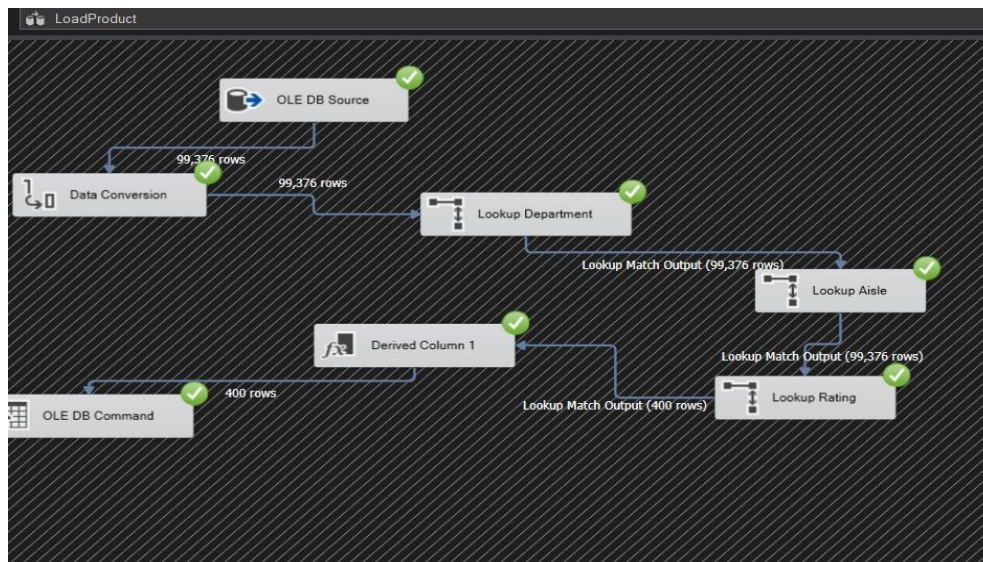


Derived column1

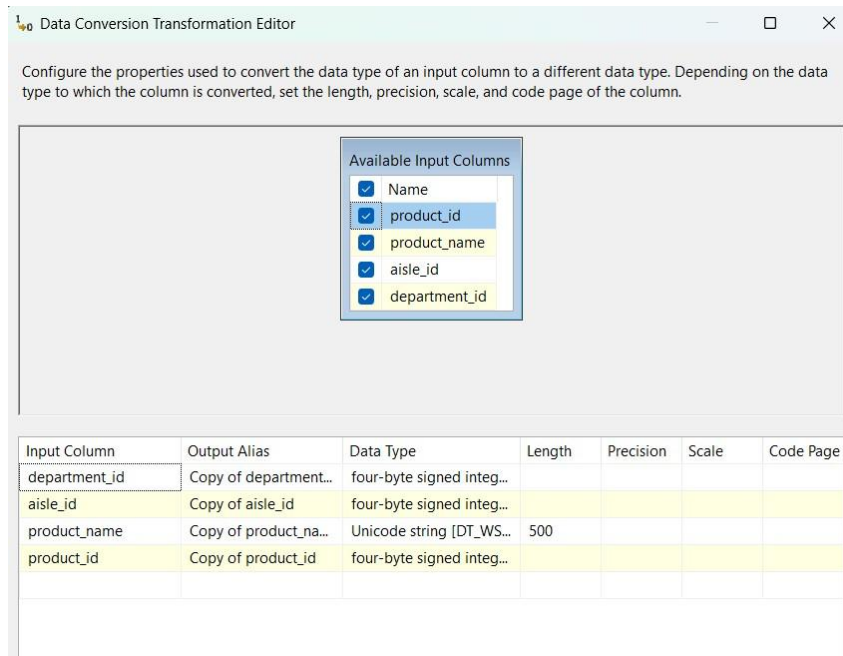
5.2.5. Transform & Load to DimProduct

The ETL process for loading data into the **DimProduct** table involves multiple **lookup transformations** to maintain referential integrity and ensure correct foreign key mapping.

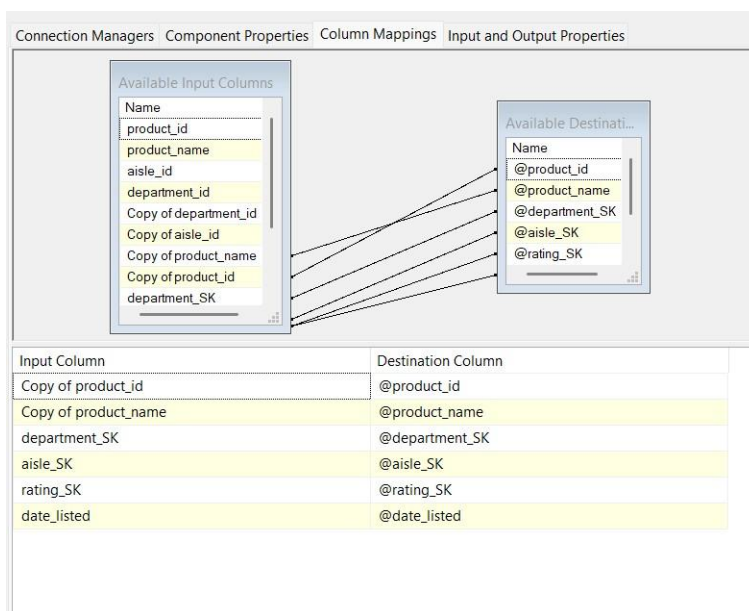
- Retrieves the corresponding **Department surrogate key** from the DimDepartment table using the department name or ID.
- Fetches the **Aisle surrogate key** by matching aisle information from DimAisle.
- Maps the product to a **Rating surrogate key** by referencing DimRating.



```
1 CREATE OR ALTER PROCEDURE dbo.UpdateDimProduct
2     @product_id INT,
3     @product_name NVARCHAR(255),
4     @department_SK INT,
5     @aisle_SK INT,
6     @rating_SK INT,
7     @date_listed DATE
8 AS
9 BEGIN
10     IF NOT EXISTS (
11         SELECT product_SK
12         FROM dbo.DimProduct
13         WHERE product_id = @product_id
14     )
15     BEGIN
16         INSERT INTO dbo.DimProduct (
17             product_id, product_name, department_SK, aisle_SK, rating_SK, date_listed
18         )
19         VALUES (
20             @product_id, @product_name, @department_SK, @aisle_SK, @rating_SK, @date_listed
21         );
22     END
23
24     ELSE
25     BEGIN
26         UPDATE dbo.DimProduct
27         SET product_name = @product_name,
28             department_SK = @department_SK,
29             aisle_SK = @aisle_SK,
30             rating_SK = @rating_SK,
31             date_listed = @date_listed
32         WHERE product_id = @product_id;
33     END
34 END;
```



Data convention



Column Mapings OLE DB command

5.2.6. Transform & Load to FactOrder

The SSIS package shown above handles the transformation and loading of data into the **FactOrders** table. This process involves multiple **lookups**, **derived columns**, and **data formatting** to ensure accuracy and consistency in the data warehouse.

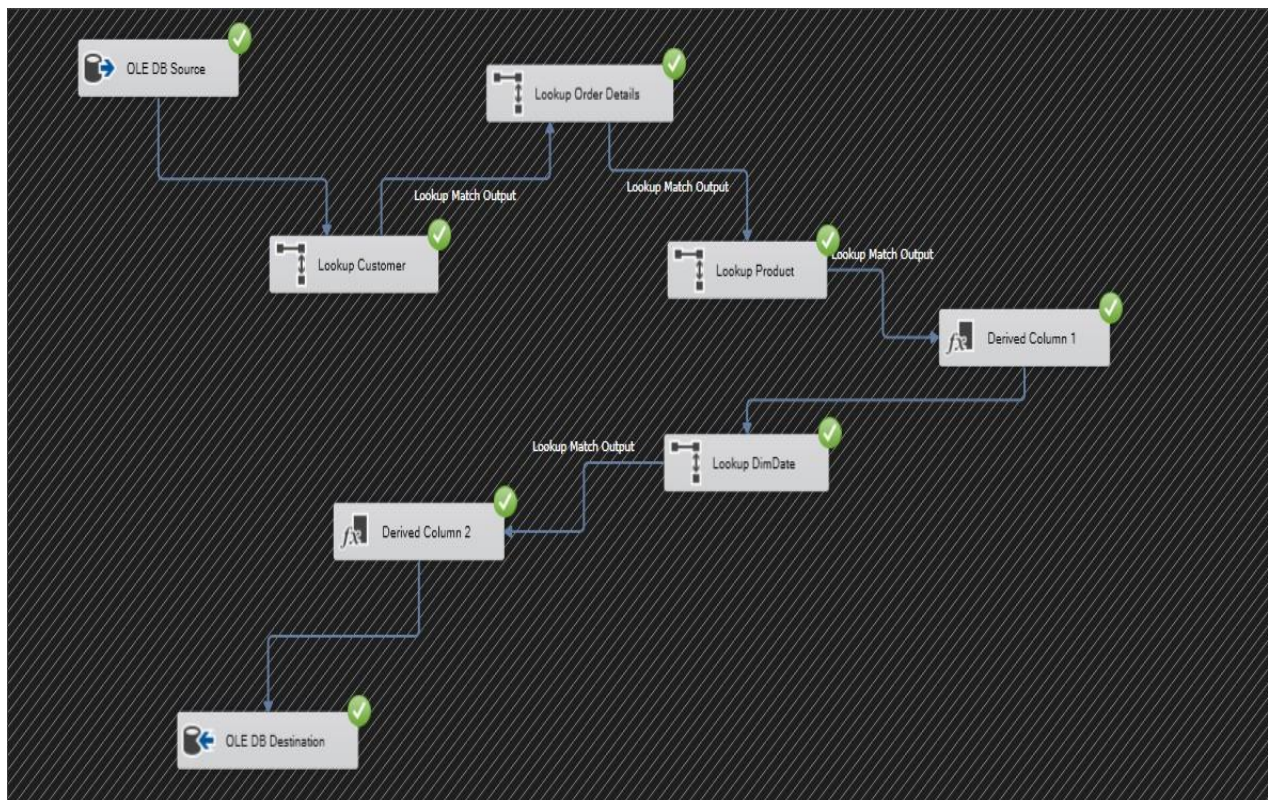
Lookup Transformations:

- **Lookup Order Details:** Matches order-specific transactional data.
- **Lookup Customer:** Retrieves the correct surrogate key from the DimCustomer table.
- **Lookup Product:** Links each order to the corresponding product in DimProduct.
- **Lookup DimDate:** Resolves the order date to its corresponding surrogate key in DimDate.

Derived Columns:

1. **Formatted Order Date** A transformation was added to format the order_date to match the format used in the **DimDate** table:

Purpose: Ensures proper matching with the DimDate table by adding leading zeroes to month and day values (e.g., converting "4/2/2025" to "04/02/2025").



- Variables and Parameters - Columns	- Mathematical Functions - String Functions - Date/Time Functions - NULL Functions - Type Casts - Operators			
Description:				
Derived Column Name	Derived Column	Expression	Data Type	Length
DateSK	<add as new column>	(DT_I4)REPLACE((DT_WSTR,10)(DT_DBDATE)or...	four-byte signed inte...	

Derived column1

Derived Column Transformation Editor

Specify the expressions used to create new column values, and indicate whether the values update existing columns or populate new columns.

- Variables and Parameters - Columns	- Mathematical Functions - String Functions - Date/Time Functions - NULL Functions - Type Casts - Operators			
Description:				
Derived Column Name	Derived Column	Expression	Data Type	Length
unit_price	<add as new column>	(DT_R8)0	double-precision flo...	
discount	<add as new column>	(DT_R8)0	double-precision flo...	
accm_txn_create_time	<add as new column>	(DT_DBTIMESTAMP)GETDATE()	database timestamp ...	
accm_txn_complete_t...	<add as new column>	NULL(DT_DBTIMESTAMP)	database timestamp ...	
txn_process_time_ho...	<add as new column>	(DT_I4)0	four-byte signed inte...	

Derived column2

6. ETL development-Accumulating Fact Tables

To create Accumulated fact table I created a new SSIS package(**Update_Accumulating_Fact**).

extended the FactOrder table by adding three new columns:

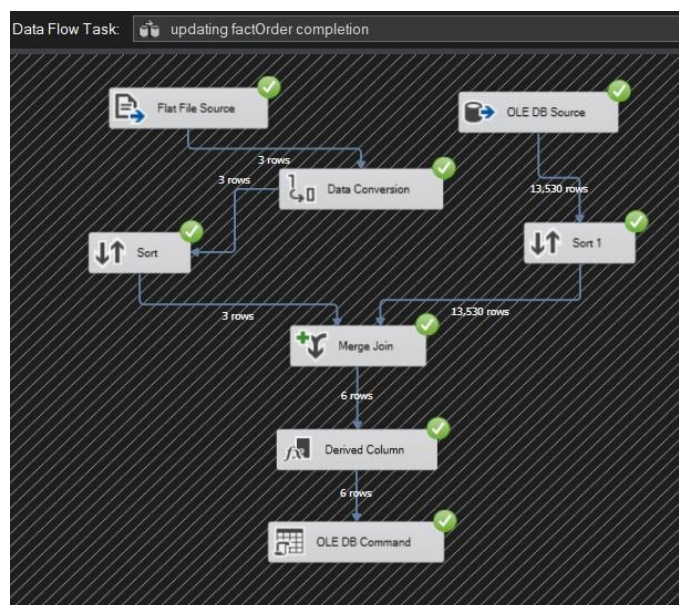
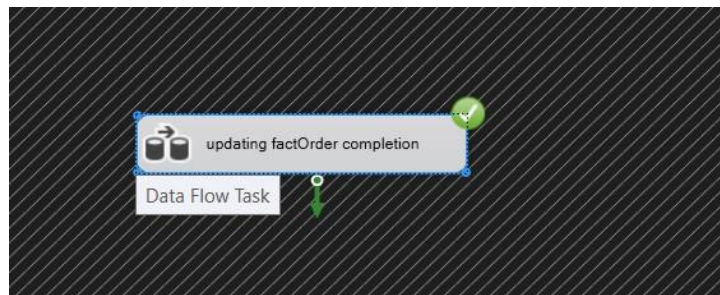
accm_txn_create_time, accm_txn_complete_time, and txn_process_time_hours.

During the initial load, I set the accm_txn_create_time using the current system date to represent when the record was created.

Then created a separate dataset containing txn_id and accm_txn_complete_time, and used a new SSIS package to update the completion time in the FactOrder table.

After that, calculated the processing time (txn_process_time_hours) as the difference between the completion time and the creation time. This helps to track how long each transaction takes to complete.

The data flow represents the update process for an **accumulating snapshot fact table**, specifically the FactOrder table. This type of fact table is used to **track the progress of a process over time** — in this case, an order's **creation and completion**.



```

CREATE OR ALTER PROCEDURE usp_UpdateTxnProcessingTime
    @order_id INT,
    @accm_txn_create_time DATETIME,
    @accm_txn_complete_time DATETIME,
    @txn_process_time_hours INT
AS
BEGIN
    UPDATE FactOrder
    SET
        accm_txn_create_time = @accm_txn_create_time,
        accm_txn_complete_time = @accm_txn_complete_time,
        txn_process_time_hours = @txn_process_time_hours
    WHERE order_id = @order_id;
END

```

	order_id	customer_SK	product_SK	DateSK	unit_price	discount	selling_price	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	3352537	57950	99	20260331	250	20	0	2026-03-31 21:01:22.730	2026-04-01 10:32:00.000	13
2	1259479	6107	10	20260331	180	10	0	2026-03-31 21:01:22.730	2026-04-02 14:32:00.000	41
3	2966570	6107	10	20260331	180	10	0	2026-03-31 21:01:06.457	2026-04-03 09:32:00.000	60
4	1939139	6107	97	20260331	320	30	0	2026-03-31 21:01:22.730	2026-04-02 18:32:00.000	45
5	190017	13406	196	20260331	150	0	0	2026-03-31 21:01:22.900	2026-04-01 11:32:00.000	14
6	1767622	13406	196	20260331	150	0	0	2026-03-31 21:01:06.457	2026-04-01 20:32:00.000	23
7	1687044	16802	34	20260331	220	15	0	2026-03-31 21:01:22.900	2026-04-08 11:32:00.000	182
8	1158768	53164	196	20260331	150	0	0	2026-03-31 21:01:22.730	2026-04-08 00:32:00.000	171
9	3218879	53164	196	20260331	150	0	0	2026-03-31 21:01:06.457	2026-04-07 10:32:00.000	157
10	2219070	53164	196	20260331	150	0	0	2026-03-31 21:01:22.730	2026-04-08 05:32:00.000	176
11	2595489	53164	196	20260331	150	0	0	2026-03-31 21:01:06.457	2026-04-08 07:32:00.000	178
12	480505	19035	130	20260331	200	5	0	2026-03-31 21:01:06.457	2026-03-31 23:32:00.000	2
13	2643446	60991	116	20260331	200	5	0	2026-03-31 21:01:06.457	2026-04-02 07:32:00.000	34
14	2428417	2277	196	20260331	150	0	0	2026-03-31 21:01:06.460	2026-04-07 03:32:00.000	150
15	1816109	2277	196	20260331	150	0	0	2026-03-31 21:01:06.460	2026-04-08 14:32:00.000	185
16	3126262	43020	196	20260331	150	0	0	2026-03-31 21:01:06.460	2026-04-06 11:32:00.000	124